COMP2611 COMPUTER ORGANIZATION
DATA REPRESENTATION

Numbers

- **Bits are able to represent both data and instruction in digital computers**

- **In this topic, you will learn:**
  - How to represent integer numbers (both signed and unsigned)?
  - How to represent fractions and real numbers?
  - What is a representable range of numbers in a computer?

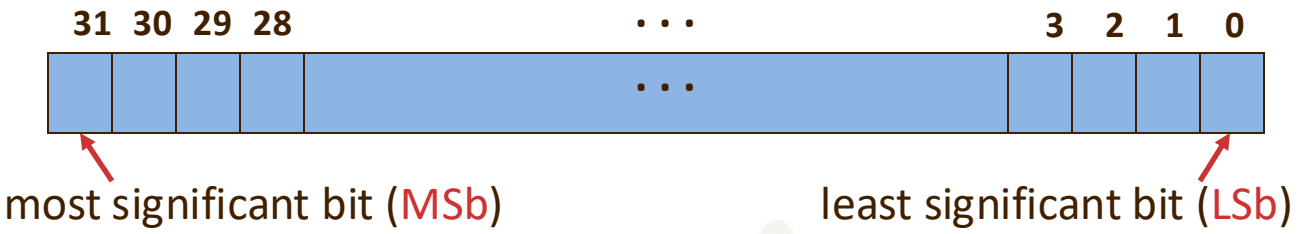
- **To be covered in Computer Arithmetic topic:**
  - Arithmetic operations: How to add, subtract, multiply, divide integer numbers
  - How to build the hardware that takes care of arithmetic operations

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

SIGNED AND UNSIGNED INTEGER

Review: Base, Representation and Value

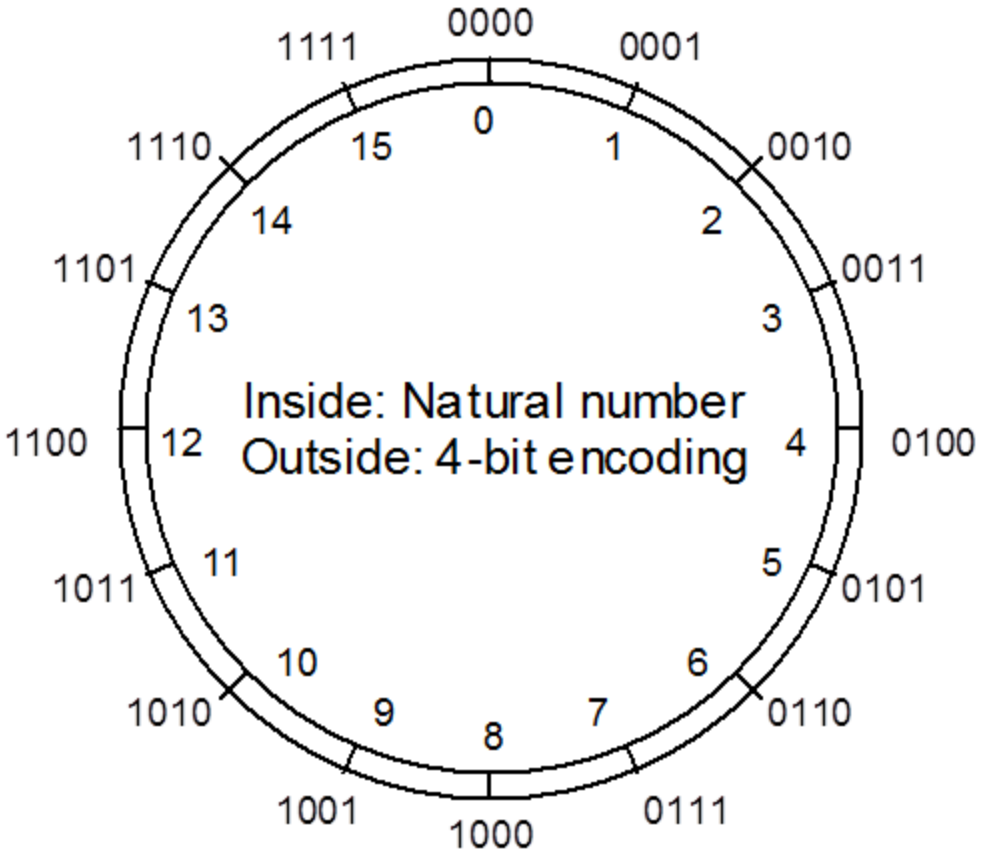
- Numbers can be represented in any **base**
  - Decimal, binary, hexadecimal
- Positional notation helps to calculate the numerical value represented
- A sequence of bits (a.k.a. **bit sequence**) usually work together
- Bits are grouped and numbered 0, 1, 2, 3 … from right to the left:



香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Unsigned Binary Integers

- Given k bits
- Min = $000.....00_2 = 0_{10}$
- Max = $111.....11_2 = 2^k - 1_{10}$
- Representable range is $[0, 2^k - 1]$



2's Complement

- **Most common scheme of representing negative numbers in computers**
- **Affords natural arithmetic (no special rules!)**
- **The most significant bit is called the sign bit**
  - 0 for non-negative, 1 for negative
- **The positive half uses the same representation as before**
- **To represent a negative number in 2's complement notation…**
  1. Decide upon the number of bits (n)
  2. Find the binary representation of the positive value in n-bits
  3. Flip all the bits (change 1's to 0's and vice versa)
  4. Add 1

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

2's Complement Example

- What is the representation of -6 in 2's complement on 4 bits?

```
i)   Start from the representation of +6
```

$0110_2 = 6_{10}$

```
ii)  Invert bits to get 1's complement
```

$1001_2 = -7_{10}$

```
iii) Add 1 to get 2's complement
```

$1010_2 = -6_{10}$

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

2's Complement Example (cont.)

- What is the representation of -6 in 2's complement on 8 bits?

i) Representation of +6	$0000\ 0110_2 = 6_{10}$
ii) Invert:	$1111\ 1001_2 = -7_{10}$
iii) Add 1	$1111\ 1010_2 = -6_{10}$

- What is the representation of -6 in 2's complement on 32 bits?

i) Start from the representation of +6

$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0110_2 = 6_{10}$

ii) Invert bits to get 1's complement

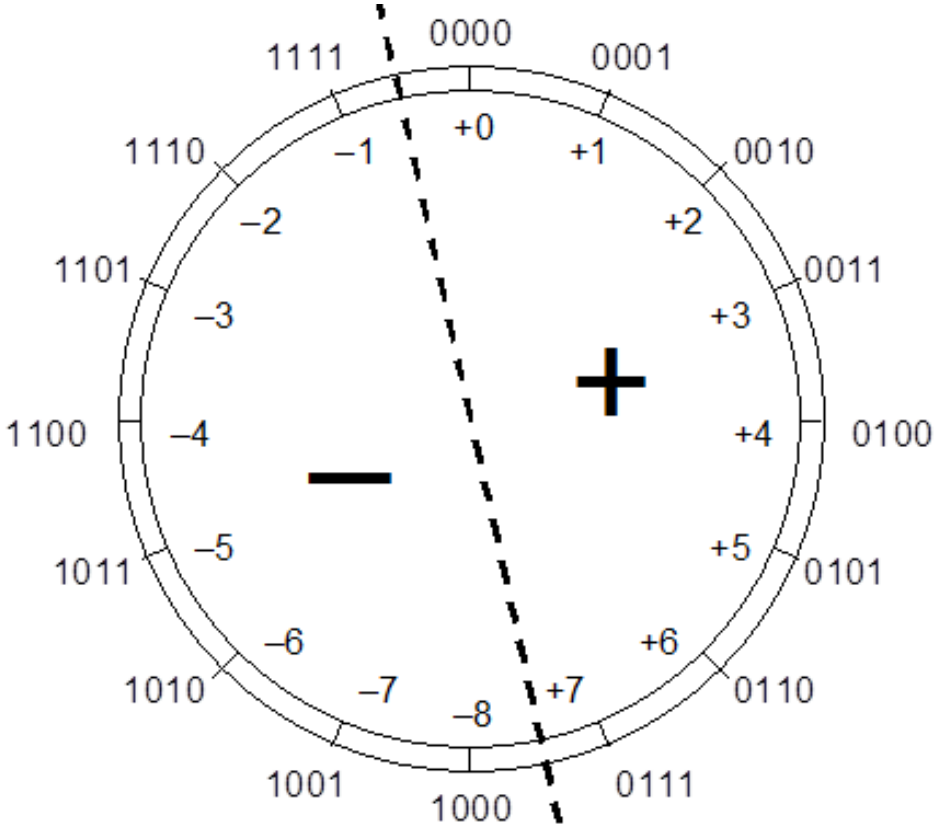
$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1001_2 = -7_{10}$

iii) Add 1 to get 2's complement

$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1010_2 = -6_{10}$

Signed Binary Integers

- **Given k bits**
- Min = $100.....00_2 = -2^{k-1}{}_{10}$
- Max = $011.....11_2 = 2^{k-1} - 1_{10}$
- **Representable range is** $[-2^{k-1}, 2^{k-1} - 1]$



Largest and Smallest Signed Integer

- Example: what is largest and smallest signed integer represented by 8 bits and 16 bits
- **largest**

$0111\ 1111_2 = 2^7 - 1 = 127$

$0111\ 1111\ 1111\ 1111_2 = 2^{15} - 1 = 32768 - 1 = 32767$

- **smallest**

$1000\ 0000_2$

Invert and add 1: $0111\ 1111_2 + 1 = 1000\ 0000_2 = 2^7 = 128$

=> - 128

$1000\ 0000\ 0000\ 0000_2$

Invert – add 1: $0111\ 1111\ 1111\ 1111_2 + 1 = 2^{15} = 32768$

=> - 32768

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

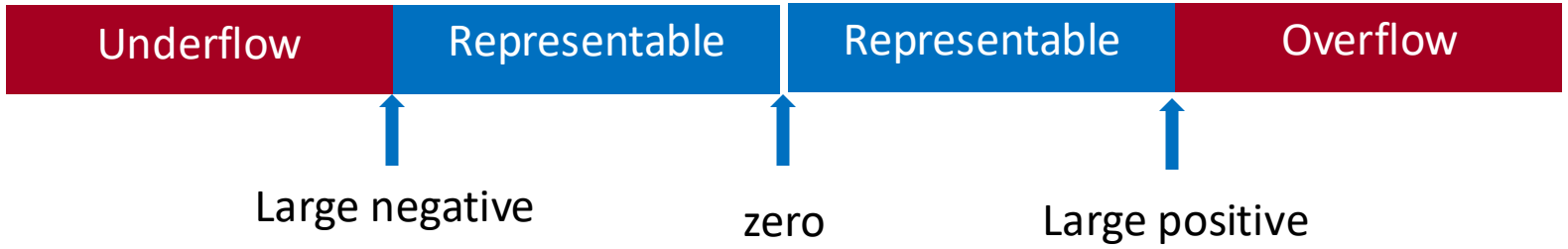
Representable Range

- **All integers in range [smallest, largest] can be represented**
- **[smallest, largest] is the representable range**

- **Largest integer represented by a 32 bit word**
  - $0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_2 = (2^{31} - 1)_{10} = 2{,}147{,}483{,}647_{10}$
- **Smallest integer represented by a 32 bit word**
  - $1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_2 = -2^{31}{}_{10} = -2{,}147{,}483{,}648_{10}$

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Overflow and Underflow in Signed Integer



- **Given the number of bits used in representing a signed integer**
  - **Overflow** (signed integer)
    - The value is bigger than the largest integer that can be represented
  - **Underflow** (signed integer)
    - The value is smaller than the smallest integer that can be represented

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Signed vs. Unsigned Integers

- **Signed**
  - **negative** or **non-negative** integers, e.g. `int` in C/C++
- **Unsigned**
  - **non-negative** integers, e.g. `unsigned int` in C/C++

- **Ranges for signed and unsigned numbers**
  - 32 bit words `signed`:
    - from

      $0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_2 = (2^{31} - 1)_{10} = 2{,}147{,}483{,}647_{10}$
    - to

      $1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_2 = -2^{31}{}_{10} = -2{,}147{,}483{,}648_{10}$
  - 32 bit words `unsigned`:
    - from

      $0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_2 = 0_{10}$
    - to

      $1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_2 = (2^{32} - 1)_{10} = 4{,}294{,}967{,}295_{10}$

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Sign Extension

- **Consider using a cast in C/C++ on a 32 bit machine**

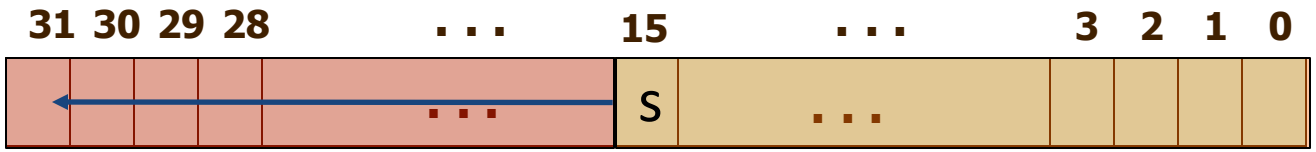
```
int i;  /* signed integer represented on 32 bits */
char a; /* Character represented on 8 bits */
i = (int)a; /* What are the values of upper 24 bits in i? */
```

- **Similar things happen in hardware when an instruction loads a 16 bit number into a 32 bit register (hardware variable)**
  - Bits 0~15 of the register will contain the 16-bit value
  - What should be put in the remaining 16 bits (16~31) of the register?

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Sign Extension

- **Sign extension is a way to extend signed integer to more bits**
  - Check the sign bit, and extend it
  - If sign is 0 then fill with 0, If sign is 1 then fill with 1



- **Example: load a 16-bit signed value to a 32-bit register**
  - 2 (16 bits -> 32 bits):

0000 0000 0000 0010 -> 0000 0000 0000 0000 0000 0000 0000 0010

  - -2 (16 bits -> 32 bits):

1111 1111 1111 1110 -> 1111 1111 1111 1111 1111 1111 1111 1110

- **Does sign extension preserve the same value?**

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Zero Extension

- **Zero extension fills missing bits with 0**
- **Example:**
  - Bitwise logical operations (e.g. bitwise AND, bitwise OR)
  - Casting unsigned numbers to larger width

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

FLOATING POINT NUMBERS

Why Floating Point?

- **Representation for non-integral numbers**
  - **Numbers with fractions** (called real numbers in mathematics)
    - e.g. 3.1416
  - **Very small numbers**
    - e.g., 0.00000000001
  - **Very large numbers**
    - e.g., 1.23456×10^{10} (a number a 32-bit integer can't represent)
- **In decimal representation, we have decimal point**
- **In binary representation, we call it binary point**

$$101.11_2 = (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) + (1 \times 2^{-1}) + (1 \times 2^{-2})_{10} = 5.75_{10}$$

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Scientific Notation & Normalized Scientific Notation

- **Scientific notation**
  - A single digit to the left of the decimal point
  - e.g. 1.23×10^{-3}, 0.5×10^{5}
- **Normalized scientific notation**
  - Scientific notation with no leading 0's
  - e.g. 1.23×10^{-3}, 5.0×10^{4}

- **Binary numbers can also be represented in scientific notation**
- **All normalized binary numbers always start with a 1**

$$1.xxx \ldots xxx_2 \times 2^{yyy \ldots yyy_2}$$

- **Example** $5.75 = 101.11_2 = 1.0111_2 \times 2^{10_2}$

- **Such numbers are called floating point in computer arithmetic**
- **Because the binary point is not fixed in the representation**

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Distribution of Floating Point Numbers

- $1.\underbrace{xxx\ldots xxx_2}_{\text{significand or mantissa}} \times 2^{\underbrace{yyy\ldots yyy_2}_{\text{exponent}}}$

- Example: 2-bit significand, exponent = {-2, -1, 0, 1}

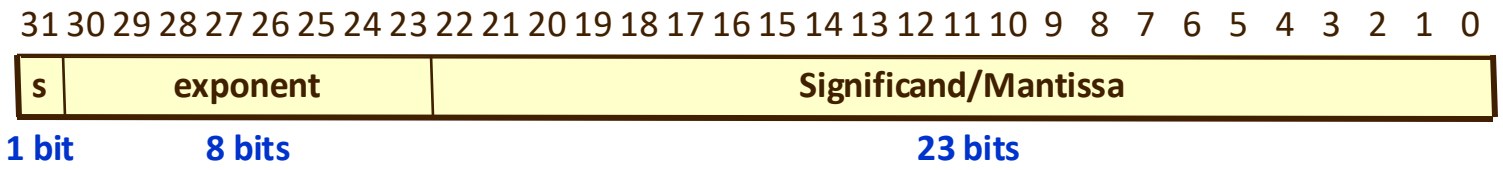
e= -2	e = -1	e = 0	e = 1
1.00 X 2^(-2)=4/16	1.00 X 2^(-1) = 8/16	1.00 X 2^0 = 16/16	1.00 X 2^1 = 32/16
1.01 X 2^(-2)=5/16	1.01 X 2^(-1) = 10/16	1.01 X 2^0 = 20/16	1.01 X 2^1 = 40/16
1.10 X 2^(-2)=6/16	1.10 X 2^(-1) = 12/16	1.10 X 2^0 = 24/16	1.10 X 2^1= 48/16
1.11 X 2^(-2)=7/16	1.11 X 2^(-1) = 14/16	1.11 X 2^0 = 28/16	1.11 X 2^1 = 56/16

- Floating point representation is approximate arithmetic
- Finite range, limited precision

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Single-Precision

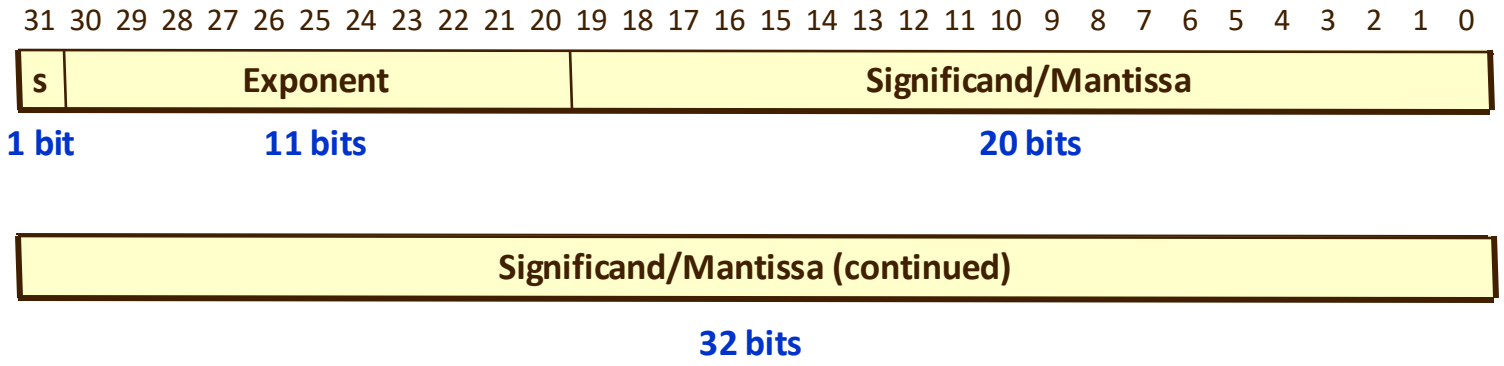
- Single-precision floating point uses **32-bit** sign-and-magnitude representation
- 8 bits for exponent, 23 bits for significand



- **Interpretation**
  - Roughly gives 7 decimal digits in precision
  - Exponent scale of about 10^{-38} to 10^{+38}

Double-Precision

- Double-precision floating-point uses 64 bits
- 11 bits for exponent, 52 bits for significand



- **Interpretation**
  - Provides precision of about 16 decimals
  - Exponent scale from 10^{-308} to 10^{+308}

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

IEEE754 Floating Point Standard

- **Defined by IEEE in 1985**
- **Developed in response to divergence of representations**
  - Portability issues for scientific code
- **Now almost universally adopted**
- **Two representations**
  - Single precision (32-bit)
  - Double precision (64-bit)

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

IEEE754 Floating-Point Format

	single: 8 bits double: 11 bits	single: 23 bits double: 52 bits
S	**Exponent**	**Significand**

$$x = (-1)^S \times (1 + Significand) \times 2^{Exponent-Bias}$$

- **Sign bit 0 ⇒ non-negative, 1 ⇒ negative**
- **Biased exponent**
  - Exponent field saves actual exponent + Bias, which ensures to be an unsigned value
  - Single: Bias = 127; Double: Bias = 1023
- **Significand with implicit 1**
  - Significand is normalized to be 1.xxxxx
  - There's always a leading 1 before binary point
  - The leading 1 is implicit (hidden bit) and not saved in significand field
- **Special cases are used to represent denormalized significand, NaN, etc.**

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Decimal -> IEEE754 Example

- **Give the IEEE754 representation of -0.75_{10} in single & double precisions**
- **Answer**
  - Scientific notation: $-0.75 = -0.11_2 \times 2^0$
  - Normalized scientific notation: $-1.1_2 \times 2^{-1}$
  - Sign = 1 (negative), exponent = -1

  - Single precision:

    S = 1, Significand = 100...00 (23 bits)

    Biased exponent = -1+127 = 01111110

    1 01111110 10000000000000000000000

  Double precision:

    S = 1, significand = 100...00 (52 bits)

    Biased exponent = -1+1023 = 01111111110

    1 01111111110 1000000000000000 000

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

IEEE754 -> Decimal Example

- What decimal number is represented by this word (single precision)?

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

- Answer:

$$(-1)^s \times (1 + Significand) \times 2^{(Exponent - Bias)}$$

$$= (-1)^1 \times (1 + 0.25) \times 2^{(129-127)}$$

$$= -1 \times 1.25 \times 2^2$$

$$= -1.25 \times 4$$

$$= -5.0$$

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Why Implicit 1 and Biased Exponent?

- **Implicit 1 packs even more bits into the significand**
  - The number is actually 24 bits long in single precision (implied 1 and 23-bit fraction), and 53 bits long in double precision (1 + 52)
- **Increase the precision of your numbers**
- **Biased exponent is for faster comparisons (for sorting, etc.), allow integer comparisons of floating point numbers**
  - Unbiased exponent
  - Biased exponent

1/2	0	1111 1111	000 0000 0000 0000 0000 0000
2	0	0000 0001	000 0000 0000 0000 0000 0000

1/2	0	0111 1110	000 0000 0000 0000 0000 0000
2	0	1000 0000	000 0000 0000 0000 0000 0000

Single-Precision Range (for Normalized Numbers)

- **Exponents 00000000 and 11111111 reserved**
- **Smallest value**
  - Exponent: 00000001
    $\Rightarrow$ actual exponent = 1 – 127 = –126
  - Fraction: 000…00 $\Rightarrow$ significand = 1.0
  - $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$
- **Largest value**
  - exponent: 11111110
    $\Rightarrow$ actual exponent = 254 – 127 = +127
  - Fraction: 111…11 $\Rightarrow$ significand ≈ 2.0
  - $\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$

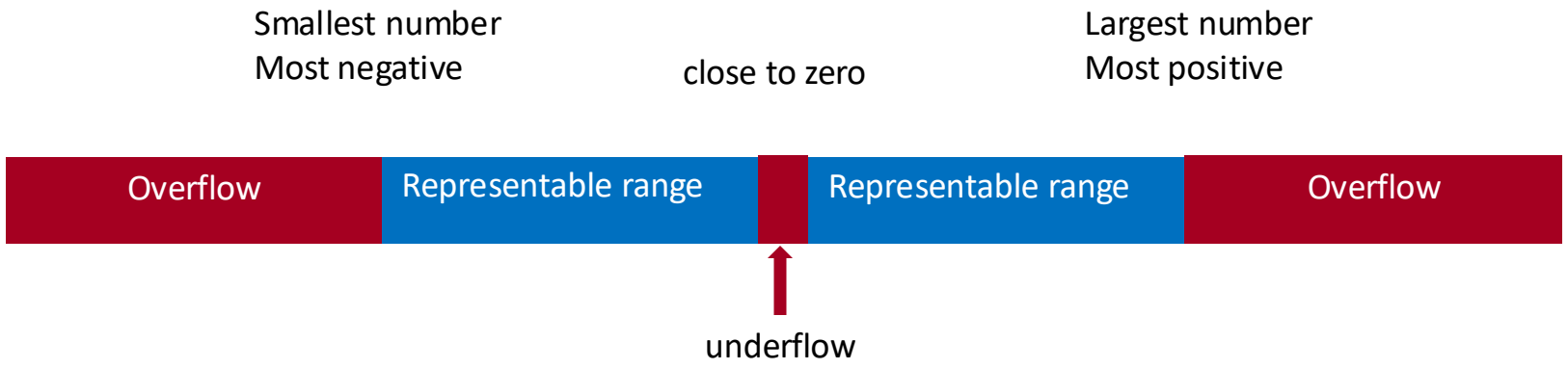
香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Double-Precision Range (for Normalized Numbers)

- **Exponents 0000…00 and 1111…11 reserved**
- **Smallest value**
  - Exponent: 00000000001
    $\Rightarrow$ actual exponent = 1 – 1023 = –1022
  - Fraction: 000…00 $\Rightarrow$ significand = 1.0
  - $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$
- **Largest value**
  - Exponent: 11111111110
    $\Rightarrow$ actual exponent = 2046 – 1023 = +1023
  - Fraction: 111…11 $\Rightarrow$ significand ≈ 2.0
  - $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Overflow and Underflow in Floating Point



- **Overflow**
  - A positive exponent becomes too large to fit in the exponent field
- **Underflow**
  - A negative exponent becomes too large to fit in the exponent field

IEEE 754 Standard for Floating Point Arithmetic

Single precision:

Denormalized | Normalized | Special Cases

Significand \ Exponent	0	1 - 254	255
0	0	$(-1)^S \times (1.F) \times (2)^{E-127}$	$(-1)^S \times (\infty)$
≠ 0	$(-1)^S \times (0.F) \times (2)^{-126}$		non-numbers e.g. 0/0 , $\sqrt{-1}$

Double precision:

Significand \ Exponent	0	1 - 2046	2047
0	0	$(-1)^S \times (1.F) \times (2)^{E-1023}$	$(-1)^S \times (\infty)$
≠ 0	$(-1)^S \times (0.F) \times (2)^{-1022}$		non-numbers e.g. 0/0 , $\sqrt{-1}$

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Denormalized Numbers and Special Cases

Exponent	Significand	Usage
111…..11	000…..00	❑ ±Infinity ❑ Can be used in subsequent calculations, avoiding need for overflow check
111…..11	≠ 000…00	❑ Not-a-Number (NaN) ❑ Indicates illegal or undefined result ❑e.g., 0.0 / 0.0 ❑ Can be used in subsequent calculations
000…..00	000…..00	❑ $\pm$0.0
000…..00	≠ 000…00	❑ Hidden bit is 0 (no implicit 1) ❑ $x = (-1)^S \times (0 + Significand) \times 2^{-126}$ ❑ Even smaller than normal number

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

IEEE754 Examples

0	00000000	00000000000000000000000 =	0
1	00000000	00000000000000000000000 =	-0
0	11111111	00000000000000000000000 =	+ infinity
1	11111111	00000000000000000000000 =	- infinity
0	11111111	01001100010001000001000 =	**NaN (Not a Number)**
1	11111111	01001100010001000001000 =	**NaN**
0	10000000	00000000000000000000000 =	$+(1.0_2)\times(2)^{128-127}=2$
0	10000001	10100000000000000000000 =	$+(1.101_2)\times(2)^{129-127}=6.5$
1	10000001	10100000000000000000000 =	$-(1.101_2)\times(2)^{129-127}=-6.5$
0	00000001	00000000000000000000000 =	$+(1.0_2)\times(2)^{1-127}=(2)^{-126}$
0	00000000	10000000000000000000000 =	$+(0.1_2)\times(2)^{-126}=(2)^{-127}$
0	00000000	00000000000000000000001 =	$+(2)^{-23}\times(2)^{-126}=(2)^{-149}$

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

CHARACTERS

Communicating with People

- Characters are unsigned bytes e.g., in C++ Char
- Usually follow the ASCII standard
- Uses 8 bits unsigned to represent a character

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

The ASCII encoding scheme

Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII
0	00000000	000	00	NUL	32	00100000	040	20	SP	64	01000000	100	40	@	96	01100000	140	60	`
1	00000001	001	01	SOH	33	00100001	041	21	!	65	01000001	101	41	A	97	01100001	141	61	a
2	00000010	002	02	STX	34	00100010	042	22	“	66	01000010	102	42	B	98	01100010	142	62	b
3	00000011	003	03	ETX	35	00100011	043	23	#	67	01000011	103	43	C	99	01100011	143	63	c
4	00000100	004	04	EOT	36	00100100	044	24	$	68	01000100	104	44	D	100	01100100	144	64	d
5	00000101	005	05	ENQ	37	00100101	045	25	%	69	01000101	105	45	E	101	01100101	145	65	e
6	00000110	006	06	ACK	38	00100110	046	26	&	70	01000110	106	46	F	102	01100110	146	66	f
7	00000111	007	07	BEL	39	00100111	047	27	‘	71	01000111	107	47	G	103	01100111	147	67	g
8	00001000	010	08	BS	40	00101000	050	28	(	72	01001000	110	48	H	104	01101000	150	68	h
9	00001001	011	09	HT	41	00101001	051	29	)	73	01001001	111	49	I	105	01101001	151	69	i
10	00001010	012	0A	LF	42	00101010	052	2A	*	74	01001010	112	4A	J	106	01101010	152	6A	j
11	00001011	013	0B	VT	43	00101011	053	2B	+	75	01001011	113	4B	K	107	01101011	153	6B	k
12	00001100	014	0C	FF	44	00101100	054	2C	,	76	01001100	114	4C	L	108	01101100	154	6C	l
13	00001101	015	0D	CR	45	00101101	055	2D	-	77	01001101	115	4D	M	109	01101101	155	6D	m
14	00001110	016	0E	SO	46	00101110	056	2E	.	78	01001110	116	4E	N	110	01101110	156	6E	n
15	00001111	017	0F	SI	47	00101111	057	2F	/	79	01001111	117	4F	O	111	01101111	157	6F	o
16	00010000	020	10	DLE	48	00110000	060	30	0	80	01010000	120	50	P	112	01110000	160	70	p
17	00010001	021	11	DC1	49	00110001	061	31	1	81	01010001	121	51	Q	113	01110001	161	71	q
18	00010010	022	12	DC2	50	00110010	062	32	2	82	01010010	122	52	R	114	01110010	162	72	r
19	00010011	023	13	DC3	51	00110011	063	33	3	83	01010011	123	53	S	115	01110011	163	73	s
20	00010100	024	14	DC4	52	00110100	064	34	4	84	01010100	124	54	T	116	01110100	164	74	t
21	00010101	025	15	NAK	53	00110101	065	35	5	85	01010101	125	55	U	117	01110101	165	75	u
22	00010110	026	16	SYN	54	00110110	066	36	6	86	01010110	126	56	V	118	01110110	166	76	v
23	00010111	027	17	ETB	55	00110111	067	37	7	87	01010111	127	57	W	119	01110111	167	77	w
24	00011000	030	18	CAN	56	00111000	070	38	8	88	01011000	130	58	X	120	01111000	170	78	x
25	00011001	031	19	EM	57	00111001	071	39	9	89	01011001	131	59	Y	121	01111001	171	79	y
26	00011010	032	1A	SUB	58	00111010	072	3A	:	90	01011010	132	5A	Z	122	01111010	172	7A	z
27	00011011	033	1B	ESC	59	00111011	073	3B	;	91	01011011	133	5B	[	123	01111011	173	7B	{
28	00011100	034	1C	FS	60	00111100	074	3C	<	92	01011100	134	5C	\	124	01111100	174	7C	|
29	00011101	035	1D	GS	61	00111101	075	3D	=	93	01011101	135	5D	]	125	01111101	175	7D	}
30	00011110	036	1E	RS	62	00111110	076	3E	>	94	01011110	136	5E	^	126	01111110	176	7E	~
31	00011111	037	1F	US	63	00111111	077	3F	?	95	01011111	137	5F	_	127	01111111	177	7F	DEL

Exercise

What does the following 32 bit pattern represent: 0x32363131

- **If it were a 2's complement integer**
- **If it were an unsigned number**
- **A sequence of ASCII encoded bytes: 2611**
  - Checking the ASCII table gives 0x32 = code for character '2', 0x36 = code for character '6', 0x31 = code for character '1', 0x31 = code for character '1'
- **A 32 bit IEEE 754 floating point number**
  - s= 0, E = 01100100, S = 01101100011000100110001
  - This is a normalized number so E is biased.

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Exercise

- **Consider building a floating point number system like the IEEE754 standard on 8 bit only, with 3 bits being reserved for the exponent.**
  - What is the value of the bias? **3**
  - What is the representation of 0? **0 000 0000**
  - What is the representation of -4?

    $-4 = -1.0 \times 2^2$

    S=1, F= 0 and the biased exponent must be

    E – 3 = 2 or E = +5

    So - 4 = 1 101 0000
  - What is the next negative value representable after – 4? **1 101 0001 = - 4.25**
  - What does the byte 1 111 1011 represent? **- NAN**
  - What is the representation of -∞? **1 111 0000**

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Concluding Remarks

- **2's complement representation for signed numbers**
  - Smallest and largest, representable range
- **Floating-point numbers**
  - Representation follows closely the **scientific notation**
  - Almost all computers, including MIPS, follow **IEEE 754 standard**
- **Single-precision floating-point representation takes 32 bits**
- **Double-precision floating-point representation takes 64 bits**
- **Overflow and underflow in signed integer and floating number**

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY